The Von Neumann Model

CSE 100/1000/1100: Introduction to Computer Engineering

Instructor:
Fatma CORUT ERGİN

Slides adapted from

- *McGraw Hill Companies, and*
- *Walid A. Najjar & Brian J. Linard*

slides

The Stored Program Computer

1943: ENIAC

- **Presper Eckert and John Mauchly -- first general electronic computer.** (or was it John V. Atanasoff in 1939?)
- **Hard-wired program -- settings of dials and switches.**

1944: Beginnings of EDVAC

- **among other improvements, includes program stored in memory**

1945: John von Neumann

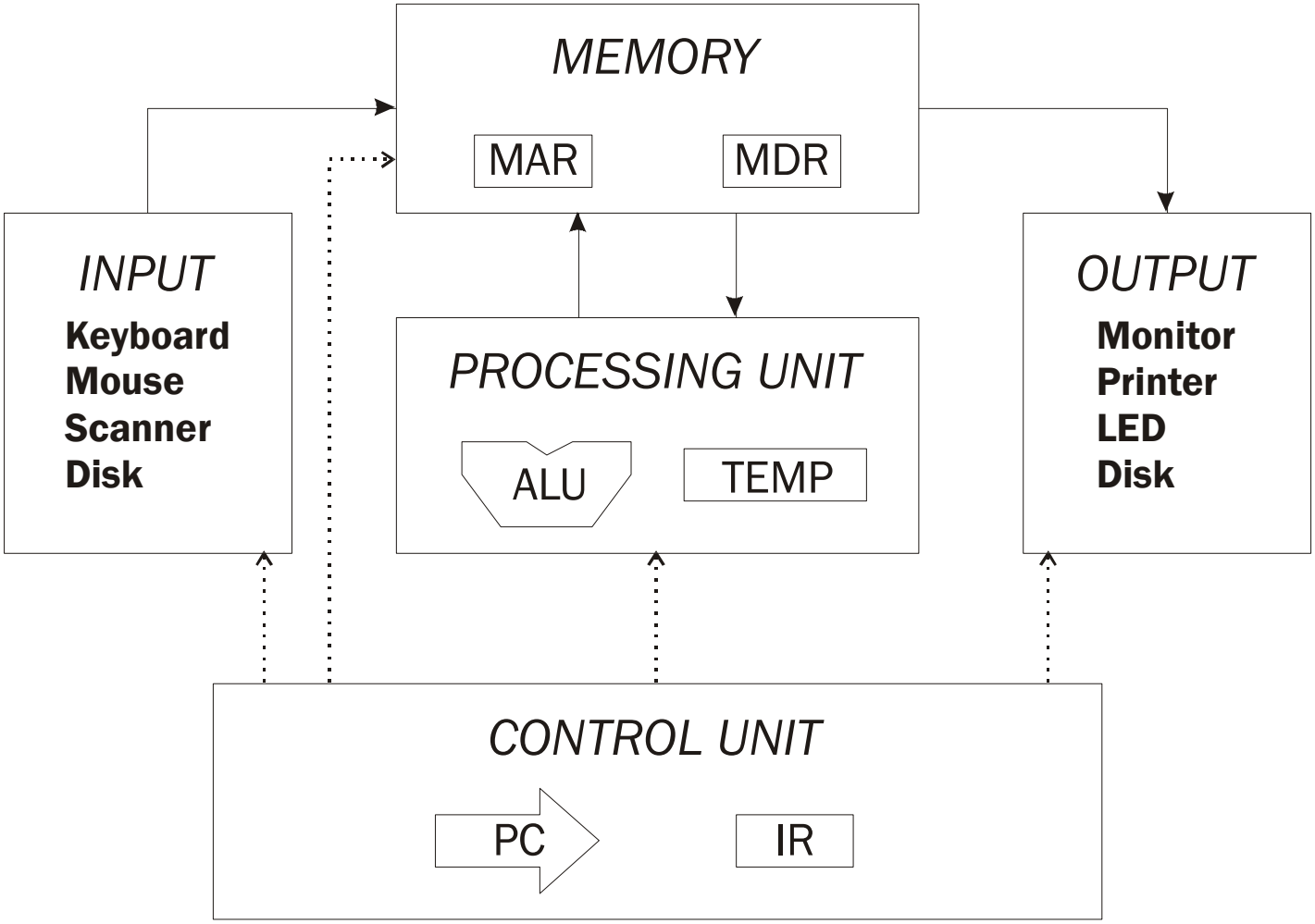
- **wrote a report on the stored program concept, known as the *First Draft of a Report on EDVAC***

The basic structure proposed in the draft became known as the "von Neumann machine" (or model).

- **a *memory*, containing instructions and data**
- **a *processing unit*, for performing arithmetic and logical operations**
- **a *control unit*, for interpreting instructions**

For more history, see http://www.maxmon.com/history.htm

Von Neumann Model



Memory

Each location has an *address* and *contents*

- ***Address:*** **bit pattern that uniquely identifies a memory location**
- ***Contents:*** **bit pattern stored at a given address.**

Address Space:

- **The total number of memory locations available.**
- **e.g. a 28 bit address provides an address space of 2^{28} locations.**

➢ *The LC-3 has an address space of 2^{16} locations*

➢ *i.e. it uses a 16 bit address*.

Memory

2^k x m array of stored bits

Address

- **unique (*k*-bit) identifier of location**
- **address space: 2^k**

Contents

- ***m*-bit value stored in location**

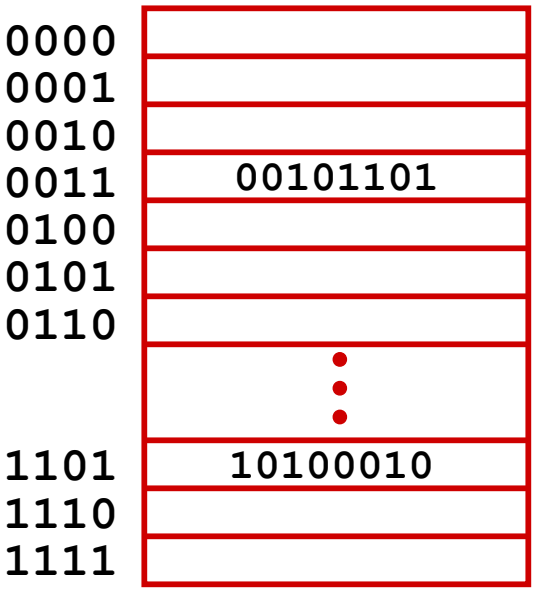
Basic Operations:

LOAD

- **read a value from a memory location**

STORE

- **write a value to a memory location**

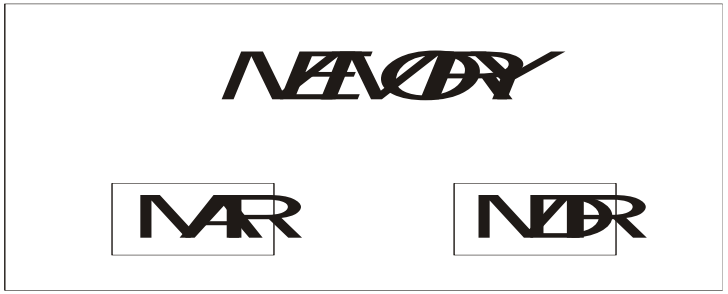


Interface to Memory

How does processing unit get data to/from memory?

MAR: Memory Address Register

MDR: Memory Data Register



To LOAD a location (A):

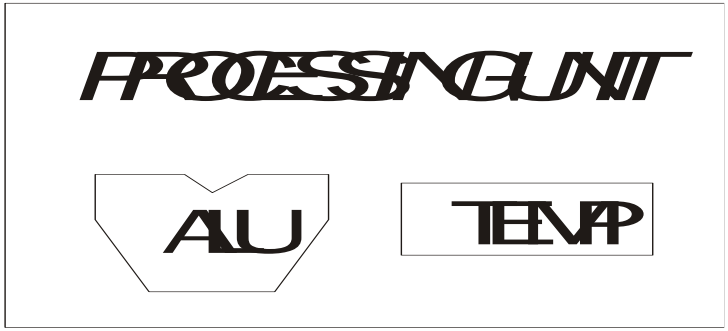
1. **Write the address (A) into the MAR.**
2. **Send a "read" signal to the memory.**
3. **Write the data into MDR.**

To STORE a value (X) to a location (A):

1. **Write the address (A) into the MAR.**
2. **Write the data (X) to the MDR (then assert Write Enable).**
3. **Send a "write" signal to the memory.**

Processing Unit

Does the actual work!



Arithmetic and Logic Unit (ALU)

- **Performs basic operations: *add, subtract, and, not*, etc.**

➢ *LC-3 performs ADD, AND, NOT*

Registers

- **Small, temporary storage**
- **Operands and results of functional units**
- **Close to the ALU (much faster access than RAM)**

➢ *LC-3 has eight registers (R0, …, R7), each 16 bits wide*

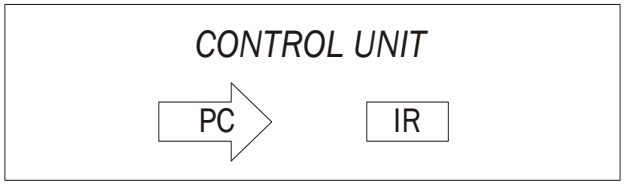
Word Size

- **number of bits normally processed by ALU in one instruction**
- **also width of registers**

➢ *LC-3 word size is 16 bits*

Control Unit

Coordinates all actions needed to execute the instruction



Instruction Register (IR) contains the *current instruction*.

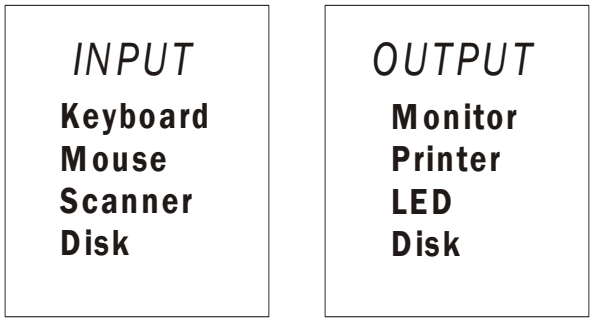
Program Counter (PC) contains the *address of the next instruction* to be executed (aka *Instruction Pointer*).

Control Unit

- **communicates with memory via the PC and IR**
- **fetches instruction from memory**
  - **the instruction's address is stored in the PC**
- **decodes the instruction, generating signals that tell the other components what to do**

Input and Output

Devices for getting data into and out of computer memory



Each device has its own interface, usually a set of registers (like the memory's MAR and MDR)

- *LC-3 supports keyboard (input) and monitor (output)*

Some devices provide both input and output

- **disk, network**

Program that controls access to a device is usually called a *driver*.

Notations

Sets of Bits

- A[3:0] denotes a set of 4 bits: A_3, A_2, A_1, A_0
- The content of an n-bit register R is referred to as R[n-1:0]
  - R_{n-1} is the most significant bit (MSB), or leftmost bit
  - R_0 is the least significant bit (LSB), or rightmost bit
  - Given R[31:0] → R[7:4] refers to the four bits from R_4 to R_7

Bit Assignment

- R[5:0] ⇐ I[13:8]
  - Means that bits 5 to 0 of register R get assigned the values of bits 13 to 8 of register I.

Contents

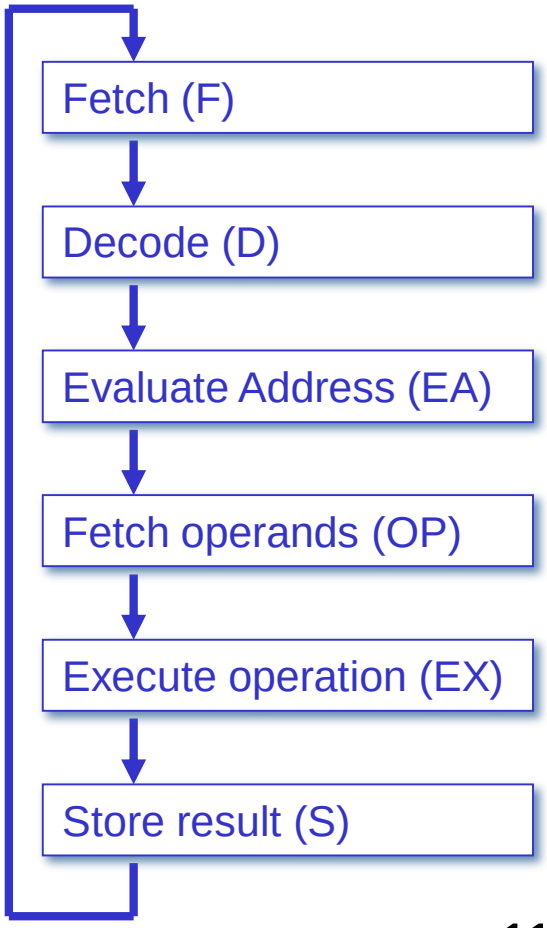
- (Reg1) means "content of Reg1"
- Mem[*loc*] means "content of memory location *loc*"

Instruction Cycle

The Control Unit orchestrates the complete execution of each instruction

Six phases of the complete Instruction Cycle

- **Fetch**: load IR with instruction from memory
- **Decode**: determine action to take
  (set up inputs for ALU, RAM, etc.)
- **Evaluate address**: compute
  memory address of operands, if any
- **Fetch operands**: read operands from memory
  or registers
- **Execute**: carry out instruction
- **Store results**: write result to destination
  (register or memory)



Instruction

The instruction is the fundamental unit of work.

An instruction is encoded as a sequence of bits.
(Just like data!)

- Often, but not always, instructions have a fixed length, such as 16 or 32 bits.
  - ***LC-3 has 16-bit instructions***
- Control unit interprets instruction: generates sequence of control signals to carry out operation.
- Operation is either executed completely, or not at all.

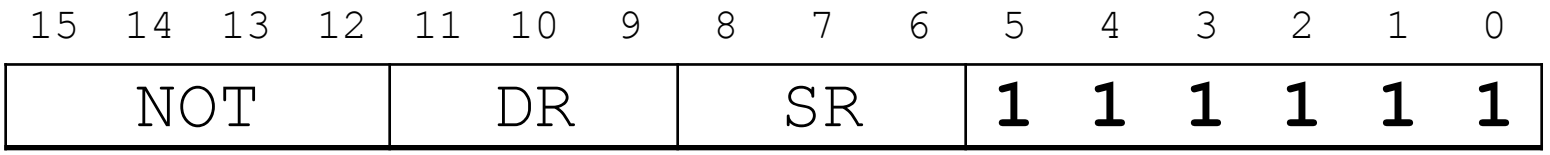
Specifies two things:

- ***opcode*: operation to be performed (*LC-3 has 4-bit opcode, bits [15:12]*)**
- ***operands*: data/locations to be used for operation**

A computer's instructions and their formats is known as its *Instruction Set Architecture (ISA)***.**

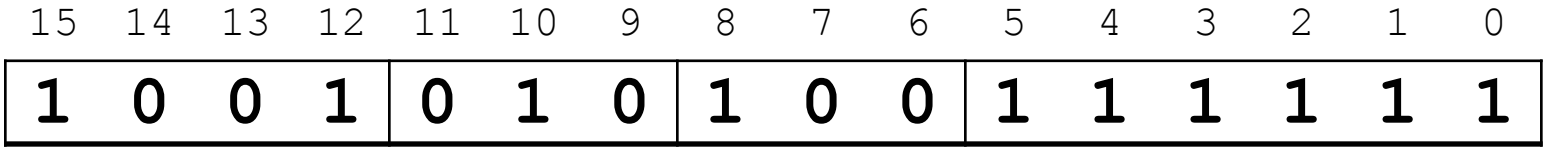
Example: LC-3 NOT Instruction

Source and destination of NOT are registers.



General Usage: **NOT DR, SR**

"Take the bitwise complement (NOT) of the contents of SR, and store the result in DR."



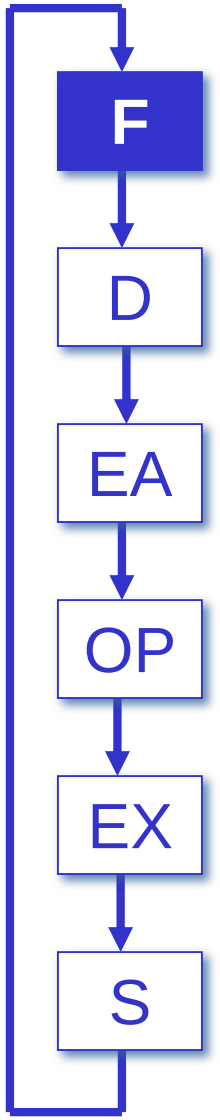
Example Instruction: **NOT R2, R4**

=> R2 = NOT(R4)

Instruction Cycle: FETCH

Load next instruction (at address stored in PC) from memory into Instruction Register (IR).

- MAR ← (PC) ; copy value of PC into MAR
- PC ← (PC) + 1 ; increment the value of PC
- MDR ← Mem[MAR] ; read memory location to MDR
- IR ← (MDR) ; copy contents of MDR into IR



- LC-3 example:
  - Assume PC= x00A1, and Mem[00A1] = x953F
  - After FETCH phase:

    MAR = x00A1, PC = x00A2, MDR = x953F, IR = x953F

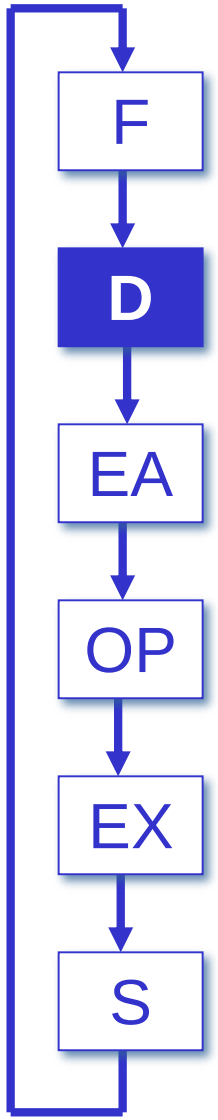
Instruction Cycle: DECODE

First, identify the opcode.

- **The opcode is input to a decoder, which sets up the following sequence of events according to the instruction.**
  - In LC-3, opcode is always the first four bits of instruction (4-to-16 decoder is used)

Depending on opcode, identify other operands from the remaining bits.

- LC-3 example: Assume IR = x953F (1001 0101 0011 1111)
  - IR[15:12] is opcode → 1001 means a NOT instruction
  - IR[11:9] is destination register → R2
  - IR[8:6] is source register → R4



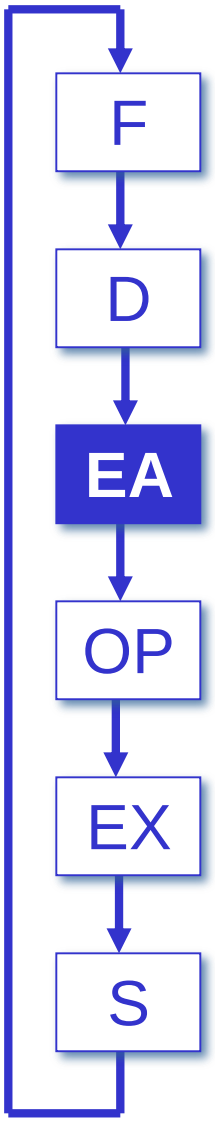
Instruction Cycle: EVALUATE ADDRESS

For instructions that require memory access, compute address used for access.

Examples:

- add offset to base register
- add offset to PC
- add offset to zero

- LC-3 NOT instruction does not require the Evaluate Address phase
- LC-3 LDR instruction requires the Evaluate Address phase

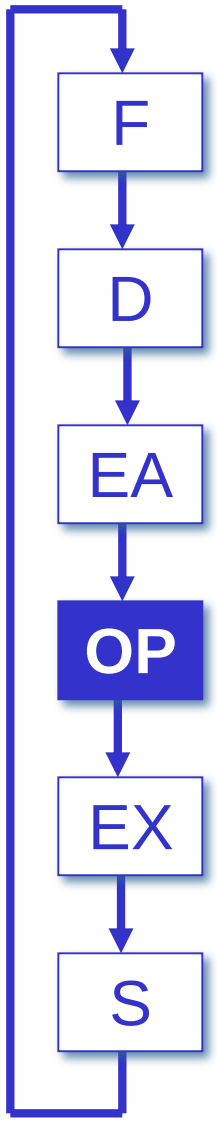


Instruction Cycle: FETCH OPERANDS

Obtain source operands needed to perform operation.

Examples:

- **load data from memory**
- **read data from register file**

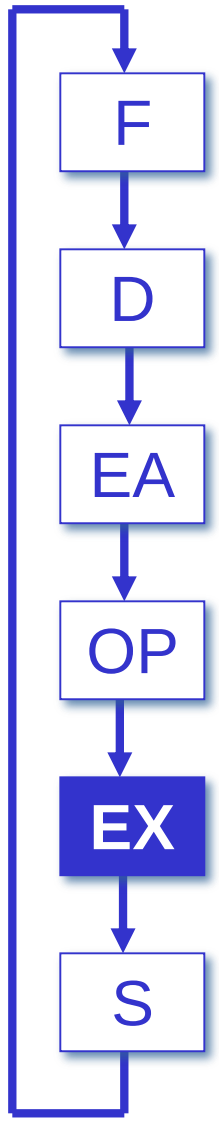


Instruction Cycle: EXECUTE

Perform the operation,
using the source operands.

Examples:

- **send operands to ALU and assert ADD signal**
- **do nothing (e.g., for loads and stores)**

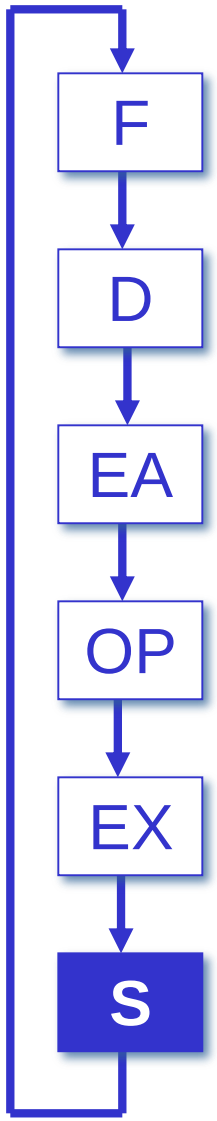


Instruction Cycle: STORE RESULT

Write results to destination.
(register or memory)

Examples:

- **result of NOT is placed in destination register**
- **result of memory load is placed in destination register**
- **for store instruction, data is stored to memory**
  - ➢ **write address to MAR, data to MDR**
  - ➢ **assert WRITE signal to memory**



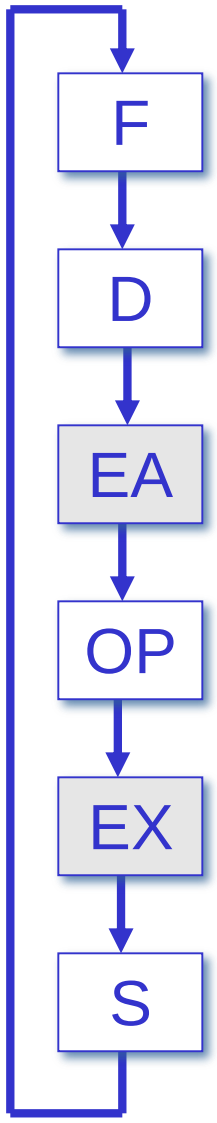
Instruction Cycle

Start over …

- **The control unit just keeps repeating this whole process.**
- **So, it now Fetches a new instruction from the address currently stored in the PC.**

Note: Some instructions don't need all 6 phases

- **If only using registers, skip Evaluate Address**
- **If only moving data, skip Execute**



Changing the Sequence of Instructions

**In the FETCH phase,
we increment the Program Counter by 1.**

What if we don't always want to execute the instruction that follows this one?

- **examples: loop, if-then, function call**

Control instructions set the PC to a new value during the Execute phase , so the next instruction comes from a different place in the program

- **jumps are unconditional -- they always change the PC**
- **branches are conditional -- they change the PC only if some condition is true**

Types of Instruction

Computational Instructions

- process data (addition, logical operations, etc.)
  - in LC-3: ADD, AND, NOT instructions

Data Movement Instructions

- move data between memory locations and registers.
- in LC-3: LD, LDI, LDR, LEA, ST, STI, STR instructions

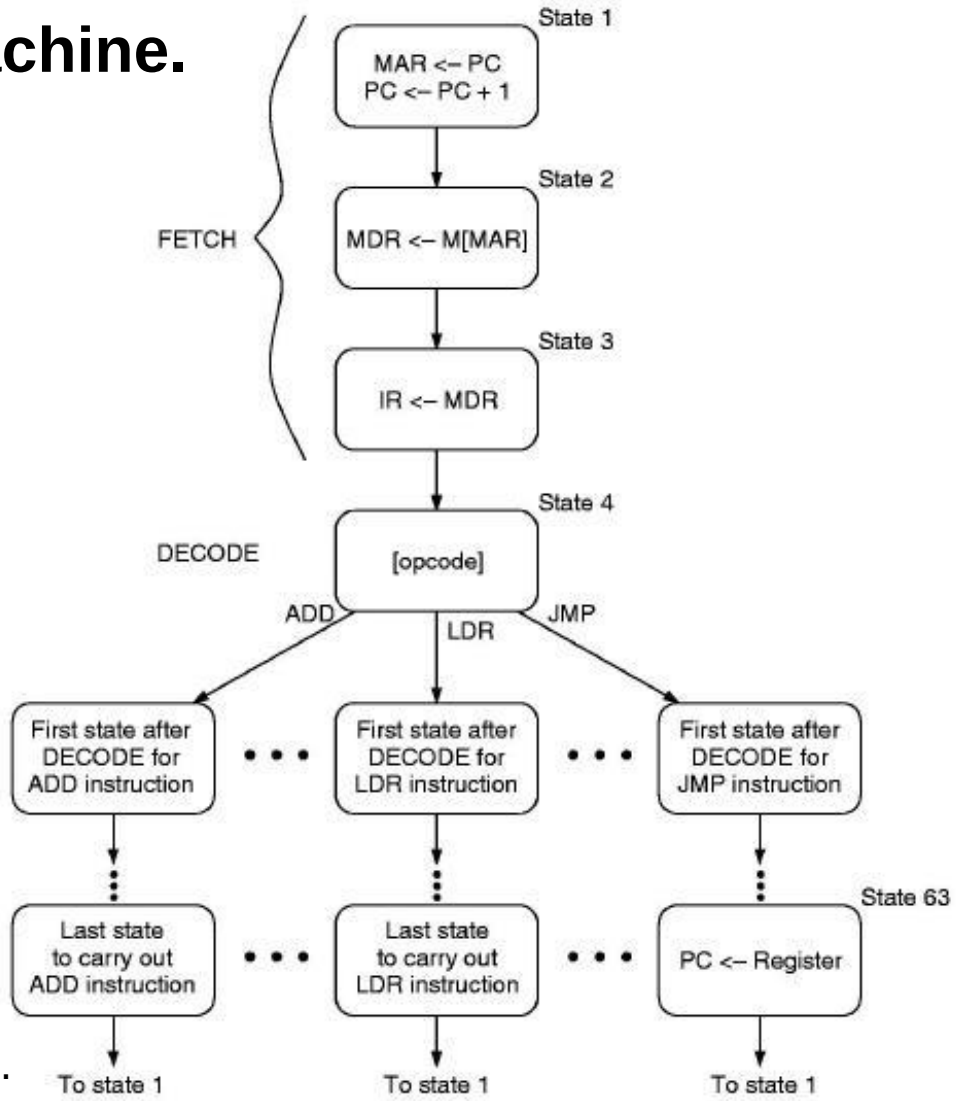
Control Instructions

- change the sequence of execution of instructions in the program
- in LC-3: BR, JSR/JSRR, JMP, RTI, TRAP instructions

Control Unit State Diagram

The control unit is a state machine.

Here is part of a simplified state diagram for the LC-3:



A more complete state diagram is in Appendix C. It will be more understandable after Chapter 5.

Stopping the Computer

Control unit will repeat instruction processing sequence as long as computer is running.

- **If not processing instructions from your application, then it is processing instructions from the Operating System (OS).**
- **The OS is a special program that manages processor and other resources.**
- **We can finish processing completely by stopping the machine cycle - i.e. by stopping the clock.**

